Zero-downtime reconfiguration mechanisms for dynamic data stream processing

Zikun Wang, Yuanli Wang, Lei Huang, Sakshi Sharma, John Liagouris, Vasiliki Kalavri {zikunw, yuanliw, lei, phsakshi, liagos, vkalavri}@bu.edu

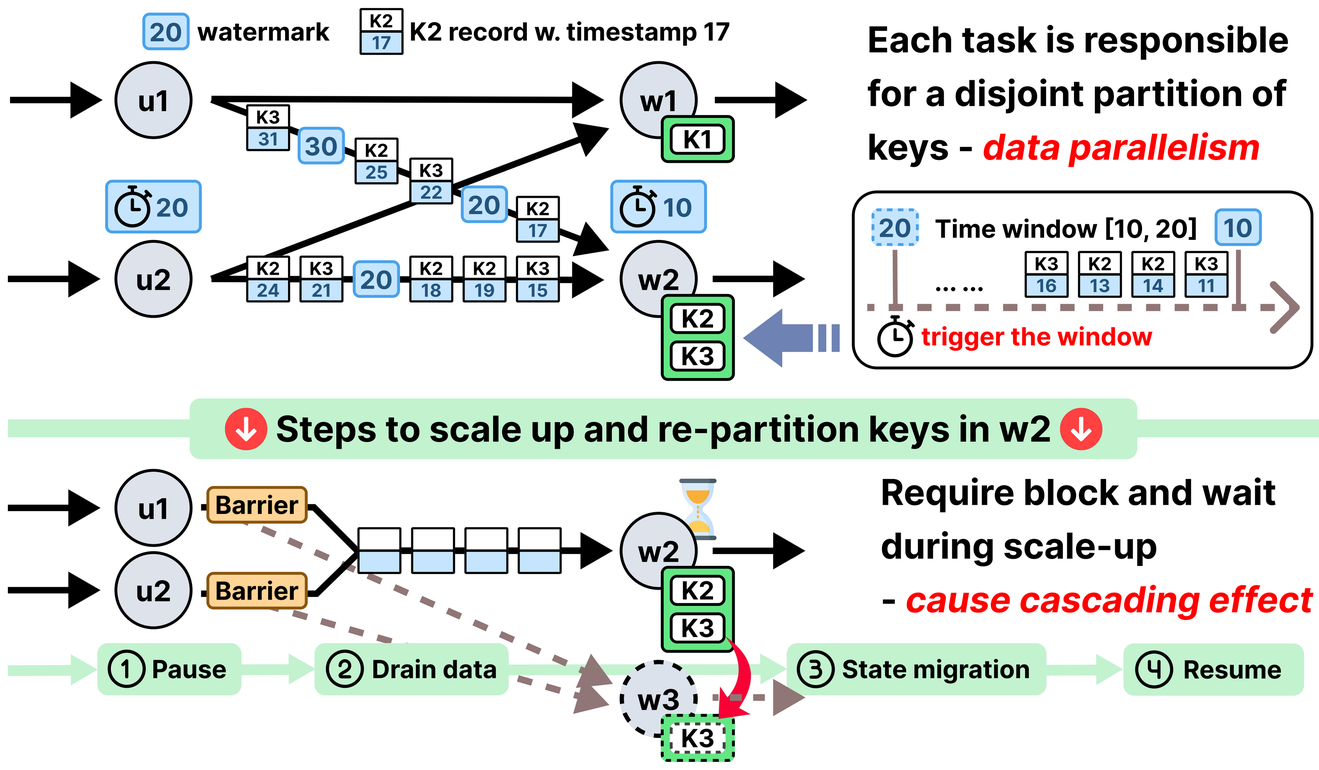
Streaming dataflow requires frequent reconfigurations

Streaming workloads are long-running, dynamic, and often unpredictable, eventually leading any initial configuration out-of-tune. Frequent reconfiguration is required.

- Rescale upon input rate changes, data skew etc..
- Task migration when original host is prohibited.

Existing mechanisms for reconfigurations:

- **Stop-and-restart**: Dataflow needs to pause execution to drain in-flight data before applying the reconfiguration.
- **State migration**: Stateful operator stores state locally for high-performance stateful processing. State migration is required during reconfigurations.



Towards a zero-downtime reconfiguration mechanism

Limitations of existing approach: System downtime

- Blocking is required to flush state and ensure the correctness.
- Modern streaming engines use local (e.g. Flink) or remote (e.g. Millwheel) storage as state backend with various ***tradeoffs***. Both require draining in-flight data.

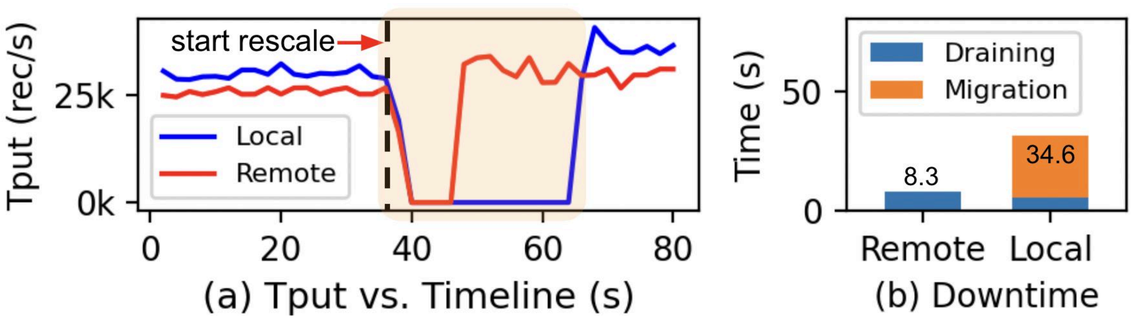


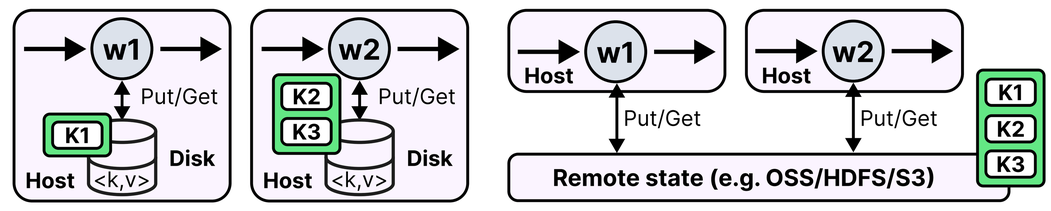
***Figure**: system downtime for a simple rescale case - re-partition key space and scale up the stateful operator from 2 to 3 replicas. State migration ≈ 10GB data.*

Local state system

- **Pros:** Low latency state access with simplified execution model. High performance.
- **Cons:** State migration overhead. Limited local storage capacity. High fault tolerance overhead.

Remote state system

- **Pros:** State migration not needed. High resource efficiency by scaling compute and storage separately.
- **Cons:** High latency state access. Long data draining time to flush local cache.



Local state **Remote state**

***Goal**: design non-blocking mechanisms to flexibly and efficiently apply reconfigurations*

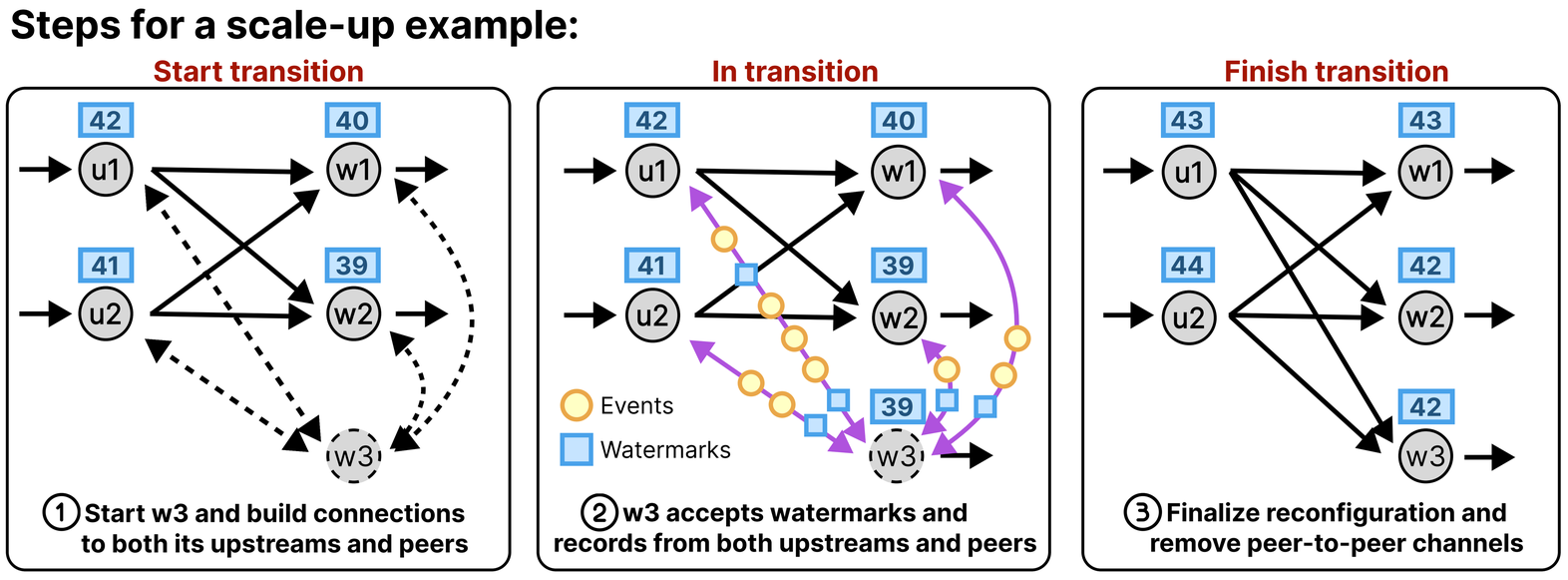
1. Apply reconfiguration decision without system downtime
2. Hybrid state access to get the benefits of both local and remote storage

State management disaggregation with lazy migration

Key idea - State management disaggregation:

- Introduce ***State Service*** to decouple state management from the dataflow.
- Freely apply reconfiguration and update dataflow topology without concerning state.
- State service is responsible for ensuring the state correctness by ***maintaining channels between replicas of the same stateful operator to flexibly and lazily transfer state during runtime***.

Steps for a scale-up example:



State migration Cases for step ②:

